Herald

The bi-directional ingesting system events and reliably fanning them out to multiple notification channels so every alert reaches the right destination without confusion.

All exiting projetc upstreams:

- GitHub: `git@github.com:vasic-digital/Herald.git` (main repository)
- GitLab: `git@gitlab.com:vasic-digital/herald.git`
- GitFlic: `git@gitflic.ru:vasic-digital/herald.git`
- GitFlic: `git@gitverse.ru:vasic-digital/Herald.git`

What Herald can (must be able to) do?

Herald is the mechanism (system) which receives input from the source(s) and sends it to one or more destinations (implemented channels). Depending on the implementation (Flavor) of the Herald sources and destinations could be various. We can have single type or multiple type inputs and same applies for the outputs.

For example, input can be the result of execution of some pipeline. Heralds is then sending this result (some report for example) to certain output (or outputs). It could be, for a sake of illustration, a messaging system which notifies subscribers.

The Possibilities are not limited!

The structure of the System MUST be hierarchical so in the top levels (closest to the root) we have abstractions and base reusable, main shared mechanisms, shared codebase, while inside the `flavors` directory we MUST have all flavors - the implementations.

Note: We MUST NOT be obligated to follow this structure as many parent project's specific custom flavors may need to exist! They could be private and require safe location in the System or project. We MUST make sure that such flexibility is possible! This MUST BE fully supported!

How Herald should achieve its mission objectives?

Every herald flavor will be individual binnary which users can add to the System path throguh the `.bashrc` or `.zshrc` (for example). Each Herald flavor will have shared set of commands and parameters while there will be as well commands and parameters specific to particular Herald flavors (implementations).

Herald applications are CLI binarries which are mainly designed for CI integration and various Pipelines. They can be easily incorporated for use with various AI CLI Agents (Claude Code, OpenCode and others...) as well or other similar use cases (triggering by Crond - cron jobs, and so on ...).

Some Herald application names can be: `pherald` for the `Project Herald`, `sherald` for the `System Herald` and so on.

Integration into the Constitution

Once the whole project is fully implemented, tested and verified with proof(s) and confirmation of complete anti-bluff validation and verification we MUST incorporate it into the root Constitution, `AGENTS.md` and `CLAUDE.md` (`constitution` Submoudule - git@github.com:HelixDevelopment/HelixConstitution.git) with the mandatory rules and constraints. Each Flavor (see below) will present its Constitution extensions!

How Constitution Submodule rules and mandatory constraints are extended

We MUST EXTEND the `constitution` Submodule with the following rules and mandatory constraints which will make possible for us to do the extending:

- Any Submodule that has the `constitutable` directory or any directory containing it inside and that is located in the root of the Project and which contains the structure and files like the `constitution` Submodule has, it will be used for extensions and overrides of the top of the definitions provided by the `constitution` Submodule.
- Rules and mandatory constraints are loaded, evaluated and applied in the following priority: `constitution` Submodule -> `constitutable` directories extensions and overrides for Constitution, `CLAUDE.md`, `AGENTS.md` and other defintions we support by the `constitution` Submodule -> Project and Submodules Constitution, `CLAUDE.md` and `CLAUDE.md` and other definition files defining rules and mandatory constraints.
- The `constitutable` directory can have multiple subdirectories with `constitution` Submodule layouts in it. For example all these paths are roots for extending or overriding `constitution` Submodule rules and mandatory constraints: `constitutable` (and all content directly in the root of the directory), `constitutable/flavor_1`, `constitutable/flavor_2`, `constitutable/flavor_3/variant_1`, `constitutable/flavor_3/variant_2`. Each MUST HAVE in itself the one of the mandatory files used for recognizing the `constitution` Submodule compatible defintions for rules and mandatory constraints: `Constitution.md`, `CLAUDE.md` or `AGENTS.md`.

Technology stack

Herald project and all flavors MUST BE writtn in Go. The whole implementation - the binary we distribute and use with all its dependencies MUST BE Containerized using the `containers` Submodule: `https://github.com/vasic-digital/containers`.

Main Database: Postgres, part of the main Container (Docker or Podman Compose stack). In-Memory Database: Redis, part of the main Container (Docker or Podman Compose stack).

All Container ports shall start with 70XXX prefix for ports so we eliminate conflicts possibility woth other containers.

So basically, we will be using ports one by one: 70001, 70002, 70003, etc.

All System (Herald) Containers names MUST start with prefix `herald`.

Commons

The following paragraphs define shared functionality and implementations among all Flavors of Herald.

The `commons` will contain the most generic abstractions and shared implementations which will be later inherited through the inheritance hierachy.

Example:

```
commons -> commons level 1 -> commons level 2 -> ... -> commons level N -> Flavor
```

Common Messaging Herald

Common Messeging Herald (`commons_messaging`) is the `commons level 1` abstraction layer. Every Messeging Herald Flavor MUST offer support for several main integrations:

- Telegram
- Slack
- Max (max.ru)
- Email
- Markdown document with PDF and HTML export

For each of Messaging services user MUST provide required tokens, credentials or API keys depending on the platform. All details MUST BE documented in proper user guides and manuals with step by step instructions so users can easily obtain and provide required information.

All sensitive data like credentials, tokens or API keys MUST be in inside proper `.env` file (create for the documentation purposes `.env.example` file to illustrate everything that users can put there) which MUST BE Git ignored! System MUST BE capable to obtain all these environment variables from exported variables from `.bashrc` and `.zshrc` or any other System profile script which can export the variables. Using values fron `.env` comes after top level ones are loaded and everything defined inside the `.env` file will override them (if same variabels are defined there).

Everything that is sent or received through any of the interated Messengers channels such as Telegram, Slack, Max and others (Email as well) will be stored inside main Markdown file and exported into PDF and HTML regularly. Markdown file and its exports MUST BE always in sync! Location of the Markdown file inside the Project MUST BE the following: `docs/herald/diary/main.md` (`main.pdf` and `main.html`).

Messaging flow(s)

Messages we send (the data) to the channels (messengers integrations) MUST BE supported to work (if particular messenger channel allows this) all of the following scenarios:

- Simple message: we send content to the channel (textual), it is sent and displayed in messenger channel to all subscribers
- Message with attachment(s): we sent the content with one or more attchments, it is sent and displayed in messenger channel to all subscribers which can then download the attchments
- Simple quote message: we send content to the channel (textual) that is a reply to an existing channel message, it is sent and displayed in messenger channel to all subscribers (it can contain zero, one or more attachments which subscribers can download)

Subscribers

Subscribers are all users added to channels of the supported Messengers. They can communicate with the System! Users (Subscribers) can receive everything the System publishes and interact! For example, to particular message contaning some information User (Subscriber) can reply, reply with an attachment or he can just brand new message with or without attachment.

Particular Flavors of Herald will have the understanding for the content received from Subscribers and about the once we send towards them (with or without attchemtns).

APIs

We MUST perform in depth research and bring int all required APIs and SDKs required for each Messaging solution to be fully incorporated. We MUST perform deep web research and obtain information about API documentation and SDKs (Go). Every SDK and API which are available as Git repositories MUST BE incorporated as Git Submodules into the project. Example path for the Submodule(s): `commons_messaging/api/telegram` or `commons_messaging/sdk/telegram`.

We MUST fully integrate Max for Business (https://max.ru and https://business.max.ru/)

Same applies for Slack and Telegram.

Priority of integration is: Telegram, Max, Slack.

For upcoming iterations we MUST document the following upcoming Messengers to be integrated: Microsoft Teams, Lark, Discord, WhatsApp, Viber. Probably some more will be added.

For each platform (Messenger integration) we MUST perform in-depth web deep research and gather all documentation, articles, technical documentation and opensourced codebases with official and unoffcial APIs and SDKs and other components we could integrate.

We MUST make sure that all materials we gather and use MUST BE properly adapted and put in our own version of technical documentation under the `docs` directory into properly structured hierarchy (SDKs, APIs, and so on).

Flavors (the implementations)

Main Go abstractions and shared codebase (with shared implementations) will be used as the base which Flavors will inherit and build on top of it.

Project Herald

The Project Herald is focused on Projects and its development. All Projects share some commons and Project Herald MUST FIT as the universal player here!

What are the specifics that Project Herald is having and others do not?

Project Flavor of Herald is focused on Projects development and all development lifecycles. Main purpose is integrating it into the development cycles and pipelines (for example LLM driven).

System during the process gets into situations, certain events happen during the development lifecycle and these information are properly organized and formulated for sending to Subscribers.

Project Flavor recognizes most common scenarios during the regular development lifecycles and in what form to communicate the events and content towards the Subscribers (Users).

Project Flawor recognizes special commands and keywords that can be part of sent content (messages) towards the Subscribers and vice versa.

Project Flavor (like all other Flavors) MUST support proper mechanisms to detect validity of messages received from Subscribers.

Our System - Project Flavor listens for messages (and responses) from the Subscribers. Every received message from Subscribers side is processed. Some of received messages do require response, some may not.

Responding to messages MUST reference the parent message in the conversation (reply / quote) if any! If it exists (parent message) IT MUS BE REFERENCED!

In processing the whole thread of communication in replies / qutes (chined replies - all from bottom to the start of the particular thread) is fully taken into the account and parsed and processed!

Serious security validation is performed before any other steps are taken!

Inputs

All data we receive from Subscribers - messages content (fresh messages or part of threads), attachments users provide, are all Inputs

Input commands

- `Bug:`, `Issue:` - Reporting problems
- `Query:`, `Request:`, `Question:` - Requesting the information or report
- Tbd

Input attachments

Tbd

Project Herald's Constitution rules and mandatory constraints

Tbd

System Herald

Tbd

What are the specifics that System Herald is having and others do not?

System Herald's Constitution rules and mandatory constraints

Tbd

Others and misc

Here we will list main ideas for upcoming Flavors which MUST BE planned with proper deep web research and fully implemented:

- Tbd

Specification documents

We MUST add into the Constitution (`Constitution.md`), `AGENTS.md` and `CLAUDE.md` of the Herald project itself the following rule / mandatory constraint related to this technical specification:

Whenever this document (`docs/specs/mvp/specification.md`) or any under the `docs/specs` root directory and any of its children directories (any level deep) is modified, comprehensive planning and implementation of all changes is MANDATORY to be performed! This does not apply to new or renamed files! For new files we MUST explicitly tell the worker (CLI agent) what to do with the newly created / copied files!

Documentation

Make sure the main `README` document is fully updated with all relevant project details and all user guides and manuals are properly linked (and other documentation and relevant materials too) to it!

We MUST have all mandatory documentation up to the smallest details, full user guides and manuals, diagrams and scheme in all major formats and other relevant materials users may need.

Testing

Whole project and all of its derrivates MUST follow testing rules from our root Constitution, `CLAUDE.md` and `AGENTS.md` (`constitution` Submodule from the main parent project).

Notes

- Many tehcnical details which can be specified for particular Herald Flavor or certain specialization may be actually general-purposed! These all MUST be identified during the processing of this specification and planned as shared (between the Flavors - Commons, or as shared Components in the System)!